AI TRAVEL PLANNER

TripMate.AI

여행지 · 날짜 · 예산 · 동행 · 취향 입력 →

AI가 Day별 일정 설계

기간

2026.06 – 2026.08

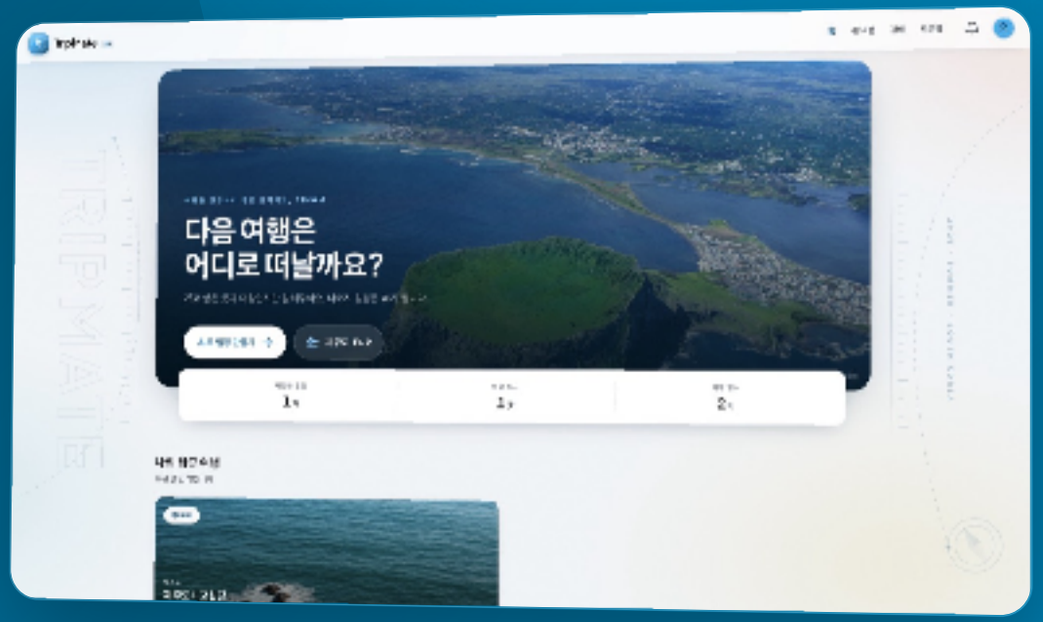
역할

기획 · FE · BE · 배포 단독

형태

팀 프로젝트 → 개인 고도화

Live Demo ↗ GitHub ↗



01 · OVERVIEW

병목은 정보 탐색이 아니라 일정 배치

문제

- 후보 장소 수집은 블로그 · 지도 · 리뷰로 이미 해결됨
- 정체 구간은 **며칠 × 시간대 × 동선** 배치
- 식사 시점, 하루 소화 가능량 판단에 시간 소모

해결

- 조건 입력을 **3단계 설문**으로 축소
- 배치는 AI 위임 — 식사 시간대 · 필수 방문지 반영
- 생성 → 저장 → 수정 → 공유까지 단일 플로우

8 화면

앱 탭 6종 + 로그인 + 공유 전용 뷰

13 컴포넌트파일

React 19 · TypeScript 전면 적용

5 API

생성 · 수정 · 좌표 · 사진 · CRUD

100 목적지

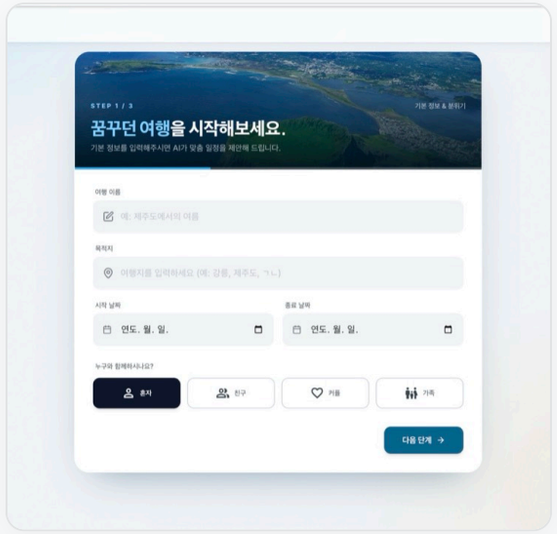
국내 자동완성 · 초성 검색

담당 범위

팀 프로젝트 종료 후 개인 저장소로 이전. **인증 · AI 파이프라인 · 배포** 전 영역 단독 고도화.

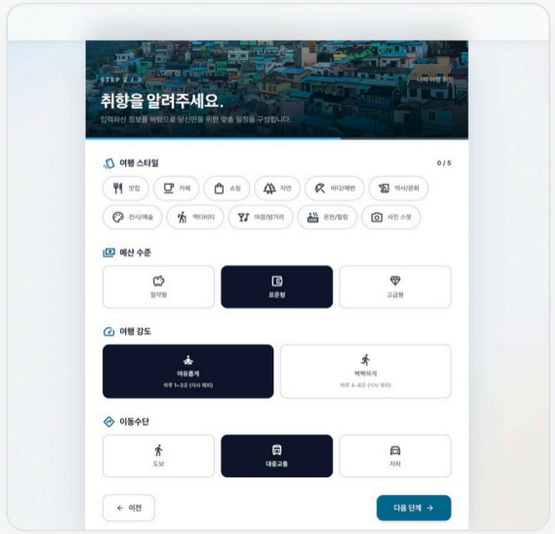
조건 입력 3단계, 결과 1화면

1 목적지 · 일정



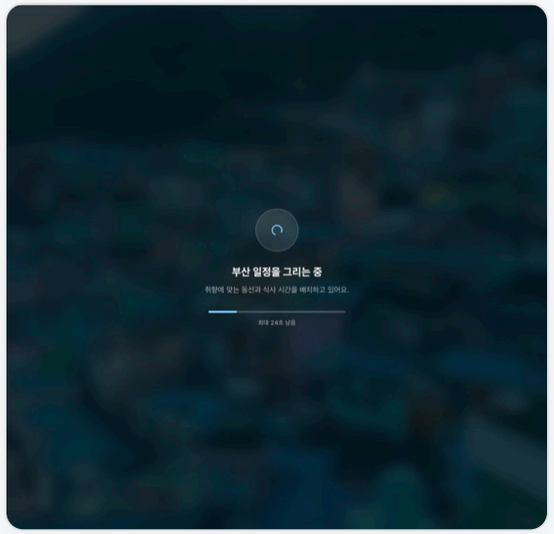
국내 100곳 자동완성. 초성 입력 대응. 날짜 역전 시 즉시 경고.

2 취향 · 조건



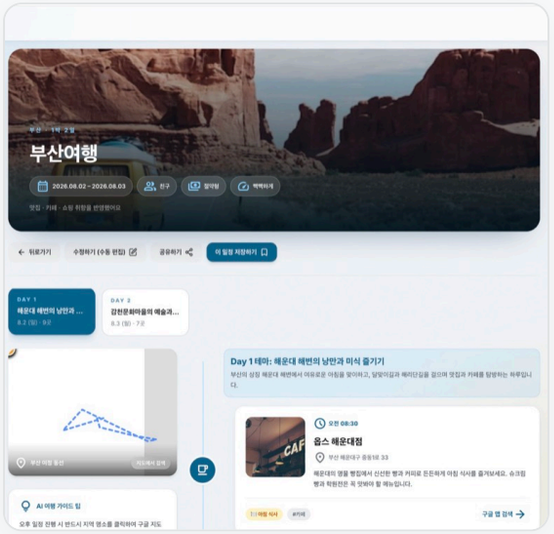
스타일 11종 중 최대 5개. 예산 · 강도 · 이동수단 선택. 강도가 하루 방문지 수를 결정.

3 AI 생성



후보 추출 → 실재 검증 → 날짜·순서·시각 배치까지 6단계. 실측 7.2초.

4 확인 · 저장



시간순 타임라인 + 동선 지도. 장소 클릭 시 구글 맵스 연결. 저장 시 보관함 이동.

핵심 기능

AI 일정 생성

Day별 · 시간대별 배치

- 장소 후보와 설명만 AI. 날짜 · 순서 · 시각은 코드가 계산
- 맛집 · 카페는 카카오 주변 검색으로 식사 시간대에 배치
- 필수 방문지 반드시 포함

자연어 수정

피드백 한 줄로 재생성

- "맛집 위주로 변경" 입력 → 기존 일정 기반 재구성
- 조건 재입력 불필요
- 시간 · 장소 · 메모 수동 편집 병행

실재 검증

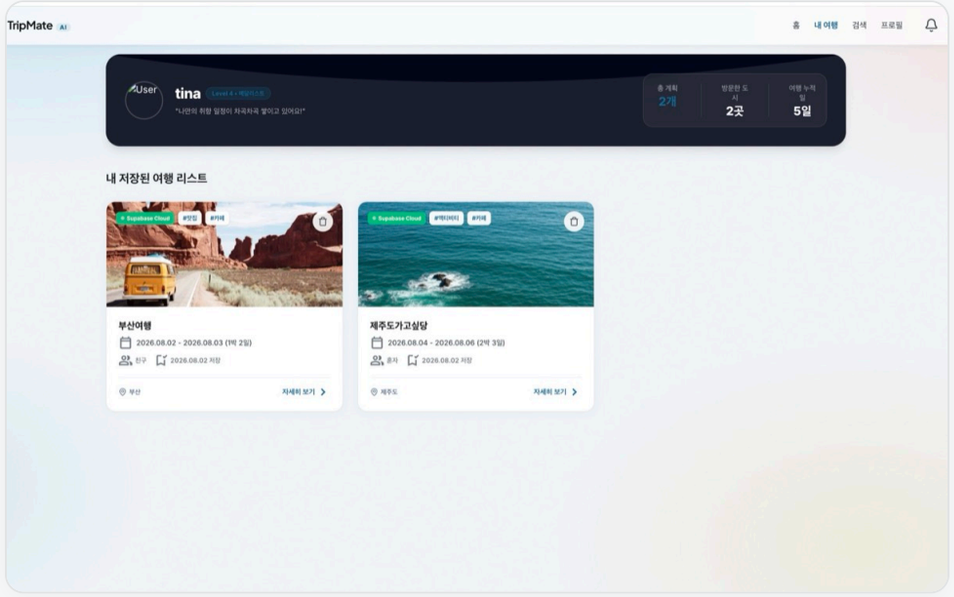
지어낸 장소를 걸러냄

- AI가 제안한 이름을 카카오 지도에 대조. 없으면 탈락
- 실좌표를 확보해 이동 시간 계산에 사용
- 하루 동선을 지도 경로로 표시

보관 · 공유

개인 보관함 · 링크 공유

- Supabase Auth 로그인 후 사용자별 저장
- 공유 설정 시 `/trip/:id` 링크 생성
- 비로그인 사용자 열람 가능



▲ 보관함 — 저장 일정 카드 목록 · 상세 뷰어 연결

04 · TECH STACK

기술 스택

FRONTEND

React 19 · TypeScript 5.8 · Vite 6 · Tailwind CSS 4 · TanStack Query · Motion · Leaflet (CDN) · lucide-react · vite-plugin-pwa

라우터 미도입. `navStack` 상태 기반 스택 내비게이션, 공유 링크만 경로 파싱.

BACKEND

Node.js · Express 4 · Netlify Functions · esbuild

로컬 Express, 운영 서버리스 함수 4종. API 키는 서버 계층에만 존재.

DATA · AI

Supabase Auth · PostgreSQL + RLS · Gemini 2.5 Flash · Flash-Lite · Kakao Local API · Google Places API

AI는 후보·설명만 담당, 응답은 JSON 스키마 강제. 실재 검증은 카카오. Places는 키 등록 시 실사진으로 전환.

품질 · 배포

Netlify CI/CD · ESLint 10 · Vitest · Lighthouse

푸시 시 자동 빌드 · 배포. 타입 검사 · 린트 · 테스트를 npm 스크립트로 고정.

도입 기준

- 한글 초성 검색 — 라이브러리 대신 유니코드 계산으로 직접 구현, **번들 증가 0**
- 라우터 — 스택 내비게이션 구조상 도입 이득 부족, 미채택. 공유 링크만 경로 파싱으로 처리
- 인증 · DB — Supabase 위임으로 AI 파이프라인에 개발 리소스 집중

API 키 브라우저 미노출 구조

CLIENT

React SPA

index.html — 앱 본체
landing.html — 랜딩 (React 미포함)

보유 정보: `/api/*` 경로 + Supabase anon key

fetch /api/* →

서버 계층 (이중 구성)

로컬 개발
`server/server.ts`
Express + Vite 미들웨어

운영
`netlify/functions/`
서버리스 함수 4종

GEMINI_API_KEY · KAKAO_REST_API_KEY ·
GOOGLE_PLACES_API_KEY 는 이 계층에서만 참조

→ 서버 전용 키로 호출

Gemini API

일정 생성 · 자연어 수정

Kakao Local API

장소 실재 검증 · 실좌표 조회 · 주변 식당 검색

Supabase

Auth + PostgreSQL
브라우저 직접 호출 — anon key + RLS로 소유권 통제

DB만 직접 호출하는 이유

- anon key는 공개 전제 설계, 실제 방어선은 RLS 정책
- 중간 프록시 계층은 요청 전달만 담당해 코드만 증가
- Gemini · Places 키는 공개 전제 아님 → 서버 경유 유지

이중 서버 구성의 트레이드오프

- 확보: 로컬 HMR 환경에서 프론트 · API 동시 기동
- 과거 비용: 프롬프트 · 스키마가 양쪽에 중복돼 한쪽만 고치는 결함 발생
- 해소: 파이프라인을 `server/itinerary/`로 추출해 양쪽이 같은 코드를 호출. Netlify 함수는 75줄, `server.ts`는 915줄에서 453줄로 축소

일정 메타데이터 · Day별 활동 분리

3단 구조

auth.users
Supabase 인증 계정

↓ 1 : N

travel_plans
일정 메타데이터 — 목적지 · 기간 · 예산 · 취향

↓ 1 : N

travel_items
Day별 개별 활동 — 시간 · 장소 · 카테고리

jsonb 단일 컬럼을 쓰지 않은 이유

- 통 JSON 저장 시 특정 활동만 부분 수정 불가
- 행 단위 분리로 수정 · 정렬 · 필터를 SQL로 처리

주요 컬럼

TRAVEL_PLANS	타입	설명
destination	text	목적지
start_date / end_date	date	여행 기간
styles	text[]	선호 스타일 키워드
must_visit_places	text[]	필수 방문 장소
is_shared	boolean	공유 링크 공개 여부

TRAVEL_ITEMS	타입	설명
day_number / sequence	int	일차 · 당일 순서
visit_time	time	방문 시각
place_name / category	text	상호명 · 관광/맛집/카페/쇼핑/숙소/이동
is_must_visit	boolean	필수 방문 여부 — 타임라인 강조 표시
image_url	text	카테고리별 이미지 — Places 키 등록 시 실사진 전환

07 · PROCESS

구현 5일, 안정화 4주

06.17
프로젝트 착수
기획 · 화면 설계 · 디자인 토큰 정의

07.06 · Day 1
환경 세팅 + 인증
Vite · TypeScript 구성, Supabase Auth 로그인/회원가입

07.10 · Day 2~5
AI 일정 생성 ~ 배포
Gemini 연동 · 결과 화면 · 반응형 · PWA · Netlify 배포

07.13 – 07.14
502/504 타임아웃 대응
Gemini thinking 모드 비활성화 — 로컬 서버 포함 2회 분할 수정

08.01
보안 · 품질 정리
테스트 계정 제거, PWA 아이콘 누락 수정, ESLint 실동작 설정, 소셜 로그인 정리

08.02
UX 고도화
홈 화면 개편, 목적지 100곳 + 초성 검색, 날짜 역전 경고

08.15 – 08.16
파이프라인 전면 교체
AI 단일 호출을 6단계로 분리. 실재 검증 · 날짜 배정 · 순서 · 시각을 코드로 이관. 가짜 일정 폴백 제거

08.17
무료 한도 · 배포 대응
AI 호출을 건당 4회에서 2회로 축소, 한도 소진을 429로 분리. 배포 환경변수 누락 복구

95 커밋 — 기능 구현은 초기 5일에 집중. 이후 작업은 **구조 재설계 · 결함 수정 · 측정**이 과반. 8월 중순 파이프라인 교체가 가장 큰 변경.

주요 기술 의사결정

ADR-6 · 신뢰도

AI 단일 호출을 6단계 파이프라인으로 분리

배경

중복 · 동선 · 실재하는 상호명을 전부 프롬프트 문장으로만 부탁하고 검증하는 코드가 없었음. 모델이 지키면 지켜지고 아니면 그대로 노출.

결정

후보 제안(AI) → 실재 검증(Kakao) → 날짜 배정 → 순서 결정 → 시각 배치 → 설명(AI). 실패 시 가짜 일정을 내보내던 폴백 제거.

근거

프롬프트 강화는 지켜졌는지 확인이 불가. 재요청 방식은 호출 수가 늘어 무료 한도(하루 20회)를 감당하지 못함.

결과

응답 16.2초 → 7.2초. 중간 단계가 전부 코드라 네트워크 대기 없음. 실패는 422 / 429로 구분해 노출.

ADR-7 · 재현성

날짜 배정 · 순서 결정을 결정적 알고리즘으로

배경

가까운 곳끼리 묶어 날짜를 나누고, 하루 안 방문 순서를 정해야 함.

결정

하루 정원을 건 k-means로 배정, 하루 8곳 이하는 완전탐색으로 최단 경로 확정(초과 시 근사). 시드는 난수 대신 최원점 선택.

근거

일반 k-means는 개수를 안 맞춰 하루 12곳 · 다른 날 2곳이 발생. 정원이 8이라 완전탐색 비용이 낮아 근사를 쓸 이유가 없음.

결과

같은 조건이면 항상 같은 일정. 알고리즘 회귀 테스트를 픽스처로 고정 가능.

ADR-8 · 호출 예산

설명 호출 배치화 + 모델 분리

배경

설명을 일차별로 호출해 3일 일정에 4회 소모. 무료 한도가 모델당 하루 20회라 일정 5건이면 소진.

결정

여행 전체 설명을 1회 배치 호출로 통합(건당 4회 → 2회). 설명만 Flash-Lite로 분리.

근거

4일치를 Flash로 받으면 14초 예산에 걸침(실측 13,970ms). 한도가 모델별로 잡혀 버킷이 분리됨.

결과

하루 처리량 5건 → 10건. 3일치 설명 1회 호출 4.6초. 검증은 일차별로 유지해 어긋난 일차만 강등.

전체 9건 상세 — GitHub · docs/ADR.md ↗

09 · INCIDENTS

주요 장애 대응

한도 소진 · 오분류

무료 한도를 건당 4회씩 소모, 429를 500으로 안내

증상

"조건을 바꿔 다시 시도해 주세요" 안내와 함께 생성 실패. 조건을 바꿔도 계속 실패.

원인

파이프라인이 일정 1건에 AI를 4회 호출해 하루 20회 한도를 5건 만에 소진. 429가 500으로 뭉뚱그려져 사용자가 할 수 없는 조치를 안내. 재시도 로직이 429를 즉시 재호출해 남은 한도를 추가 소모.

조치

설명 호출을 배치로 묶어 4회 → 2회. 429 전용 에러로 분리하고 리셋 시각을 한국시간으로 환산해 안내(서머타임 반영). 타임아웃 · 쿼터는 재시도 대상에서 제외.

결과

한도 소진 시 25초 예산을 태우지 않고 0.55초에 429 반환.

로컬 정상 · 배포만 실패

배포본만 생성 500 — 환경변수 누락

증상

로컬은 정상, 배포 데모만 500. 응답이 1.36초에 반환 — 서버 예산 25초의 근처에도 못 감.

진단

키 이름 불일치와 함수 타임아웃을 차례로 의심했으나 모두 반증. 1.36초라는 응답 시간이 "느린 단계에 도달조차 못 했다"는 근거였음.

원인

Netlify에 `KAKAO_REST_API_KEY` 미등록. 실재 검증 통과 장소가 0건이 되어 이후 단계 진행 불가. 파이프라인 교체로 새 필수 의존성이 생겼는데 배포 환경에 미반영.

시사점

실패 응답에서도 소요 시간이 진단 정보. 에러 문구("잠시 후 다시")가 원인을 가릴 수 있음.

배포 전 차단 · 무증상

정규식이 한글을 전부 제거

증상

중복 제거용 이름 정규화가 한글 입력을 빈 문자열로 반환. 에러 없음. 영문 입력으로는 정상 동작해 드러나지 않음.

원인

`\W`는 ASCII 기준 정의라 한글을 단어 문자로 보지 않음. `u` 플래그로도 확장되지 않음. 그대로 배포됐다면 모든 한국 장소가 동일 키로 묶여 하루 1곳만 남음.

조치

제거 대상 열거 대신 남길 대상 지정(`[^\p{L}\p{N}]`). 괄호 표기 혼재 문제까지 함께 해소.

시사점

계획서에 적힌 코드가 검증된 코드는 아님. 옮기기 전에 실제 값으로 1회 실행.

전체 12건 상세 — GitHub · docs/TROUBLESHOOTING.md ↗

성능 · 품질 실측

AI 일정 생성 응답 시간

7.2 초

파이프라인 교체 후 실측 · 부산 2박3일

구간	측정값
교체 전 (단일 호출)	16.2s
교체 후 (6단계)	7.2s
설명 배치 1회 (3일치)	4.6s
한도 소진 시 429	0.55s

서버 예산 25초 · 클라이언트 제한 30초 이내. 교체 전은 4건 평균, 교체 후는 1건 실측이라 참고치.

Lighthouse · 랜딩 (데스크톱)

Accessibility	100
Best Practices	96
SEO	82
Performance	61

CORE WEB VITALS	측정 / 목표	
TBT	0ms / <200ms	충족
FCP	2.8s / <1.5s	미달
LCP	4.7s / <2.5s	미달

주 요인은 단일 청크의 파싱 · 실행 시간. Motion 동적 import 분리가 우선 대상. **이 수치는 2026.08.03 구 파이프라인 시점 측정이며 재측정 예정.**

번들 크기

app.js	210.5KB / 300KB	충족
app.css	11.9KB / 50KB	충족
landing.html	5.8KB / 150KB	충족

전부 gzip 기준. 원본 app.js 709.2KB · app.css 73.9KB. 예산은 충족하나 단일 청크 크기가 LCP 지연 요인. 2026.08.19 빌드 실측.

코드 품질

단위 테스트 251개 통과 · **파일 18개** · **tsc --noEmit 통과** · **ESLint 실동작 설정**

7월 11개 → 현재 251개. 파이프라인 각 단계와 쿼터 · 재시도 경로까지 커버. `any` 37곳은 `no-explicit-any` 를 warn으로 유지해 추적.

측정 조건 — 응답 시간 · 테스트 · 번들은 2026.08.17~19 실측(프로덕션 빌드 로컬 실행). Lighthouse만 2026.08.03 구 파이프라인 시점 값이라 재측정 대상. 모바일 쓰로틀링 수치는 환경 편차로 미채택.

개선 과제

구조 개선 방향

AI 응답 처리를 스트리밍 / 비동기 큐로 전환

현재는 동기 호출 유지 + 예산 재분배 방식. 같은 제약에 네 차례 대응이 발생했고, 근본 해결은 설계 변경 폭이 커 의도적으로 보류.

Netlify 함수 타임아웃과 서버 예산 정합

서버 예산은 25초인데 `netlify.toml`에 함수 타임아웃 설정이 없음. 실측 11.9초라 여유는 있으나 예산 설계상 모순.

`users` / `profiles` 테이블 통합

분리 근거가 코드로 확인되지 않으며 `profiles.eamil` 오타 컬럼 잔존. 초기 스캐폴딩 확장의 결과.

인지된 제약 · 미해결 항목

- **설명 배치 호출의 길이 한계** — 7일 이상 일정은 응답이 길어져 일부 일차가 최소 문구로 강등될 수 있음
- `any` **37곳** — Gemini 응답 · Supabase 반환값 경계에 집중. 응답 스키마 타입 정의로 대부분 해소 가능
- **800줄 상한 초과 2건** — `PlanResultView.tsx` 920줄, `PlannerFlow.tsx` 812줄. `server.ts`는 915줄 → 453줄로 해소
- **E2E 테스트 부재** — 단위 테스트 251개는 확보했으나 브라우저 플로우 자동 검증은 없음
- **중복 테이블 마이그레이션 보류** — 실사용 데이터 누적으로 전환 비용 검토 중

운영 원칙으로 정리한 항목

실재하지 않는 일정을 그럴듯하게 보여주지 않는다. 8월 초에는 "폴백을 플래그로 표시"였으나, 플래그는 개발자만 보고 사용자는 화면에 뜬 일정을 믿는다는 판단으로 폴백 자체를 제거하고 실패를 상태 코드로 노출하는 쪽으로 바꿨다.

— 12 · LINKS

TripMate AI

구현 · 성능 측정 · 장애 대응 · 기술 의사결정 기록을 채널별로 분리 관리 중

Live Demo ↗

tripgather.netlify.app

회원가입 후 일정 생성 즉시 확인 가능

GitHub ↗

github.com/jjssspark/TripMate

README · 아키텍처 · API · DB · 성능 · ADR · 트러블슈팅 · 회고

Notion ↗

트러블슈팅 12건 · ADR 9건 · 회고

현상 · 원인 분석 · 시도한 방법 · 최종 해결 · 배운 점